

入力資格状態制御による長期LLMエージェント の記憶・学習・推論整合性

— 制御プレーン設計・実装アーキテクチャ・評価規律 —

Akihito Sunagawa

2026年5月22日

要旨

長期的に運用されるLLMエージェントでは、記憶を増やすこと、継続学習を行うこと、長い文脈を保持することが、そのまま性能向上につながるとは限らない。むしろ、引用、相槌、貼り付け文、第三者情報、保存禁止、削除済み情報、過去値、現在値、未確認仮説、矛盾した更新が同じ経路で扱われると、誤記憶、旧値混入、保存禁止情報の再利用、依存関係の破綻、長時間文脈での推論劣化が起きる。

本稿は、これらを個別の失敗ではなく、入力情報の資格状態、記憶状態、時点、使用権限、更新経路、矛盾解消状態の混同として定式化する。提案する枠組みを Input Qualification Control、略して IQC と呼ぶ。IQC では、入力には直ちに本人事実、現在方針、学習材料、行動根拠へ昇格しない。入力は、出所、発話行為、時点、使用権限、版状態、矛盾状態を持つ状態付き対象として保持され、記憶参照、継続学習更新、長時間文脈推論、行動候補生成の前に制御される。

本稿は、内部実装名および内部管理名を用いず、制御原理と匿名化された評価結果のみを報告する。評価は、入力資格判定、長期記憶参照、継続学習更新、矛盾蓄積による文脈劣化の四領域で行った。現行の memory-control runtime では、匿名化された長期記憶評価ゲートで 1412/1412 fixtures および 103/103 quality gates を通過した。ここで fixture は入力、読出し、削除、scope、leak などの回帰ケース単位であり、quality gate は複数fixture群を束ねた合格条件である。この評価には、自然会話stress、多言語会話、実ログ風長文、promotion holdout、read/write consistency、scope leakage、削除・session/day、model-facing readout leak、live answer gate が含まれる。selected readout の合成イベント契約では selected readout 1.000、false route 0、value miss 0 を得た。readout契約比較では、state-control reference構成が 1.0 であるのに対し、partial baseline は 0.2、naive promote-all は 0.0 まで落ちた。conflict-aware readout でも state-control reference構成 1.0 に対し、naive baseline は 0.0 だった。制御ブリッジ評価では 20/20、複数ゲート横断smokeでは 8/8 を

通過した。長時間文脈評価では、矛盾あり・制御あり条件が、矛盾あり・制御なし条件に対して大きく改善し、矛盾なし条件に近い事実想起と規則適用を維持した。

本研究は、長期記憶、継続学習、推論劣化を一般に完全解決したと主張しない。また、任意の実ログ、任意のモデル、任意の運用条件への完全一般化も主張しない。主張するのは、これらに共通する主要な失敗源を状態混同として扱い、制御可能な設計問題へ変換できること、およびテストされた長期記憶・読出し・制御ブリッジ・矛盾蓄積条件では、状態制御面を置く方向が強く支持されることである。したがって本稿の結論は、「すべてを解いた」ではなく、「長期運用LLMエージェントにおいて、この制御面を置くことが第一候補になるだけの実装証拠がある」である。

1 目的

本稿の目的は、長期LLMエージェントにおける記憶、学習、推論の失敗を、別々の機能問題ではなく、状態制御の問題として整理することである。

本稿の主張は、長期LLMエージェントの主要な失敗は「情報を忘れること」だけではなく、「情報を誤った資格で使うこと」によって生じる、という点にある。

従来の説明では、長期記憶は「過去情報を検索できるか」、継続学習は「新しい情報を学べるか」、推論劣化は「長い文脈でも正しく答えられるか」と分けて語られやすい。しかし、長期運用エージェントで実際に危険になるのは、検索、学習、推論の前に、その情報をどの資格で使ってよいか壊れることである。

以下では、この制御を入力資格状態制御、または Input Qualification Control、略して IQC と呼ぶ。

本稿では、内部実装名や内部管理名を出さない。読者に必要なのは、特定の実装名ではなく、次の四点である。

- どの失敗モードを対象にしているか。
- どの状態変数でそれを制御するか。
- どの評価条件で効果を見たか。
- どこまで主張し、どこから先は未解決とするか。

読み方として、本稿は弱い仮説メモではない。すでに複数の実装ゲート、長期記憶 stress、readout 契約比較、制御ブリッジ、長時間文脈評価が回っており、少なくとも本稿が対象にした事故クラスでは、状態を明示しない長期記憶・読出し設計へ戻る理由は弱い。一方で、本稿は外部の独立 judge、任意自然会話、任意実運用ログまで

閉じた最終標準でもない。この二つを分けるため、以下では「完全解決」ではなく「強い方向支持」として議論する。

2 最小失敗例

長期記憶や継続学習の危険は、極端な攻撃でだけ起きるわけではない。次のような短い入力でも、単純な会話要約、素朴なRAG、最新値優先メモリ、通常の継続学習更新では、誤った昇格や旧値混入が起きうる。

例1: 引用を本人事実にしてしまう

入力: 「友人が『私は退職したい』と言っていた」

失敗: ユーザー本人の退職意向として記憶してしまう。

必要な制御: 発話内容を、本人由来の現在利用可能前提ではなく、第三者情報および引用として扱う。

例2: 短い相槌を全文同意にしてしまう

入力: AIが長文で性格や方針を推測した後、ユーザーが「そう」とだけ返す。

失敗: 直前の長文全体に対する明示同意として記憶してしまう。

必要な制御: 「そう」を弱い応答または同意範囲不明として扱い、固定的な本人事実へ昇格しない。

例3: 処理対象を将来記憶に混ぜてしまう

入力: 「これは記憶しないで。以下を翻訳して」

失敗: 翻訳対象の本文を、将来利用可能なユーザー記憶へ混ぜてしまう。

必要な制御: 保存禁止と一回限りの処理対象を分離し、翻訳後の読出し、更新、行動利用から除外する。

例4: 更新後に依存知識が古いまま残る

入力: 「本社は東京です」から後日「本社は大阪に移転しました」へ更新される。

失敗: 本社だけ大阪になり、最寄駅、管轄地域、配送条件などの派生知識が東京のまま残る。

必要な制御: 新旧の版状態と依存関係を分離し、現在値更新、過去値保持、派生知識の再評価を別経路で扱う。

これらは、単なる検索精度の失敗ではない。近い文を探せるかどうかより先に、その入力が本人事実なのか、引用なのか、処理対象なのか、過去値なのか、保存禁止なのか、未解決矛盾を含むのかを判定する必要がある。

3 問題設定

長期LLMエージェントでは、入力是一个の文章であっても、複数の意味層を持つ。

- 内容: 何が書かれているか。
- 出所: 誰の情報として受け取るか。
- 発話行為: 断定、質問、相槌、引用、依頼、訂正、削除要求のどれか。
- 時点: 現在値、過去値、予定、期限切れ、未確認のどれか。
- 権限: 回答、記憶、共有、行動、学習更新に使ってよいか。
- 整合性: 既存記憶、派生知識、方針、上位制約と矛盾しないか。

問題は、これらをすべて「文脈」または「ユーザーが言ったこと」に潰してしまう点にある。

本稿では、次の三つの失敗を同じ問題族として扱う。

- 記憶参照の失敗: 引用、相槌、保存禁止、削除済み情報、第三者情報を現在利用可能な本人事実として使う。
- 継続学習更新の失敗: 現在値、過去値、削除値、禁止値、未確認仮説を同じ更新経路へ混ぜる。
- 長時間文脈推論の失敗: 未解決の矛盾が蓄積し、後続の事実想起、規則適用、矛盾検出が劣化する。

この観点では、矛盾と整合性は補助的な話題ではない。矛盾は、長期記憶、継続学習、推論劣化をつなぐ中心的な状態である。

4 中心仮説

本稿の中心仮説は次である。

長期LLMエージェントの安定性は、記憶量や文脈長だけでは決まらない。入力情報の資格状態、記憶状態、版状態、使用権限、更新経路、矛盾解消状態を分離し、それぞれの用途に応じて制御することで、誤記憶、旧値混入、保存禁止情報の再利用、依存関係の破綻、矛盾蓄積による文脈劣化を抑制できる。

最小の制御面は、次のように表せる。

生入力

-> 入力資格判定

- > 状態付き記憶候補
- > 昇格 / 保留 / 履歴化 / 除外 / レビュー
- > 選択読出し
- > 回答 / 行動 / 学習更新

ここで、記憶を保存することと、記憶を使ってよい状態へ昇格することは異なる。保存は監査や復元のために必要になりうるが、保存された情報が将来の回答、行動、学習更新に使えるとは限らない。

5 状態モデル

本稿で扱う状態は、入力イベントの最小スキーマと、台帳・更新経路で後から付与される下流状態に分けられる。

形式的には、各入力を次の状態付き対象として扱う。

```
x = (  
  content,  
  source,  
  speechAct,  
  timeState,  
  permission,  
  versionState,  
  conflictState  
)
```

ここで content は文字列や観測内容そのものであり、残りの成分は、その内容を誰の情報として、どの時点の情報として、どの用途に使うべきかを表す。speechAct には、引用、弱い相槌、依頼、訂正、削除要求だけでなく、必要に応じて同意範囲の不明確さも含める。長期記憶や継続学習で危険なのは、content だけを保存し、他の成分を落としてしまうことである。

一方、memoryState や updateRoute は、入力そのものの性質ではなく、入力資格判定の後に台帳または更新候補へ付与される下流状態である。たとえば同じ入力でも、no-store であれば処理後に保存候補から除外され、時点更新であれば旧値は archived、新値は current、派生知識は review または needs-repair へ送られる。このため、以下の表は入力イベントの軸と下流台帳の軸を同じ一覧に置くが、実装上は段階を分けて扱う。

状態群	代表値	制御する失敗
入力資格状態	本人記述、引用、貼り付け、相槌、第三者情報、外部情報、仮説	誤昇格
記憶状態	current、past、archived、erased、review、conflict、no-store、secret	旧値混入、削除済み情報の再出現
版状態	現在値、過去値、候補値、無効値、派生値	新旧混線
使用権限	回答可、行動不可、共有不可、学習不可、確認要求	保存禁止情報や秘密情報の再利用
更新経路	現在値更新、過去値保持、境界訓練、除外、レビュー	継続学習の混線
矛盾解消状態	未解決、解消済み、条件分岐、時点更新、レビュー要求	矛盾蓄積劣化、依存関係破綻

重要なのは、これらを一つの分類ラベルにしないことである。同じ内容でも、引用か本人事実か、現在値か過去値か、使用可か使用不可か、未解決矛盾か解消済みかによって、後続処理は変わる。

5.1 状態ラベル仕様

外部評価で再現性を持たせるには、状態ラベルを説明語ではなく、入力条件、許可経路、禁止経路を持つ仕様として定義する必要がある。最小仕様は次のように表せる。

軸	ラベル	入力条件	許可経路	禁止経路
source	user	ユーザー本人の断定または明示訂正	current候補、review	third-party扱い
source	third-party	他者の発言、属性、意向	参照、境界訓練	本人事実、本人行動

軸	ラベル	入力条件	許可経路	禁止経路
speechAct	quote	引用符、伝聞、転載	引用として保存、要確認	本人方針、現在値
speechAct	weak-ack	短い相槌、範囲不明の同意	hold、review	全文同意、学習更新
speechAct	task-payload	翻訳、要約、分類などの処理対象	一回限りの処理	将来記憶、学習更新
permission	no-store	記憶しない、保存しないという明示指示	直後の処理	将来読出し、学習、共有
permission	secret	秘密、鍵、認証情報、機密メモ	安全な一時処理、拒否	外部行動、共有、学習
versionState	current	現在値として明示された値	selected readout、更新	archived値との混同
versionState	archived	上書き済み、過去値、履歴	履歴参照、監査	現在回答、行動根拠
memoryState	erased	削除要求または無効化済み	監査上の存在確認	回答、行動、学習
conflictState	open	未解決矛盾、競合仮説	review、候補保持	現在前提、確定更新
conflictState	resolved	時点、条件、権限で整理済み	条件付き読出し	未解決扱いのまま混在

この表は完全な分類体系ではない。重要なのは、各ラベルが単なる保存用メタデータではなく、後段の回答、行動、記憶更新、継続学習更新を許可または禁止する制御信号になる点である。

6 矛盾と整合性

本稿では、整合性を単一の正解率として扱わない。長期運用エージェントで必要な整合性は少なくとも五種類ある。

整合性	問い	典型的な失敗
出所整合性	誰の情報として使っているか	第三者発言を本人事実にする
時点整合性	current と past を分けているか	古い住所を現在住所として答える
依存整合性	root 更新が派生知識へ反映されるか	本社更新後も配送条件が古い
権限整合性	no-store や secret を用途別に守るか	保存禁止情報を後で回答に混ぜる
矛盾整合性	未解決矛盾を現在前提として使わないか	互いに矛盾する規則を同時に適用する

このため、ベンチマークも単一の overall accuracy だけでは不十分である。事実想起、依存整合性、更新成功、過去値保持、権限違反、false route、value miss、矛盾検出を分けて見る必要がある。

7 制御アーキテクチャ

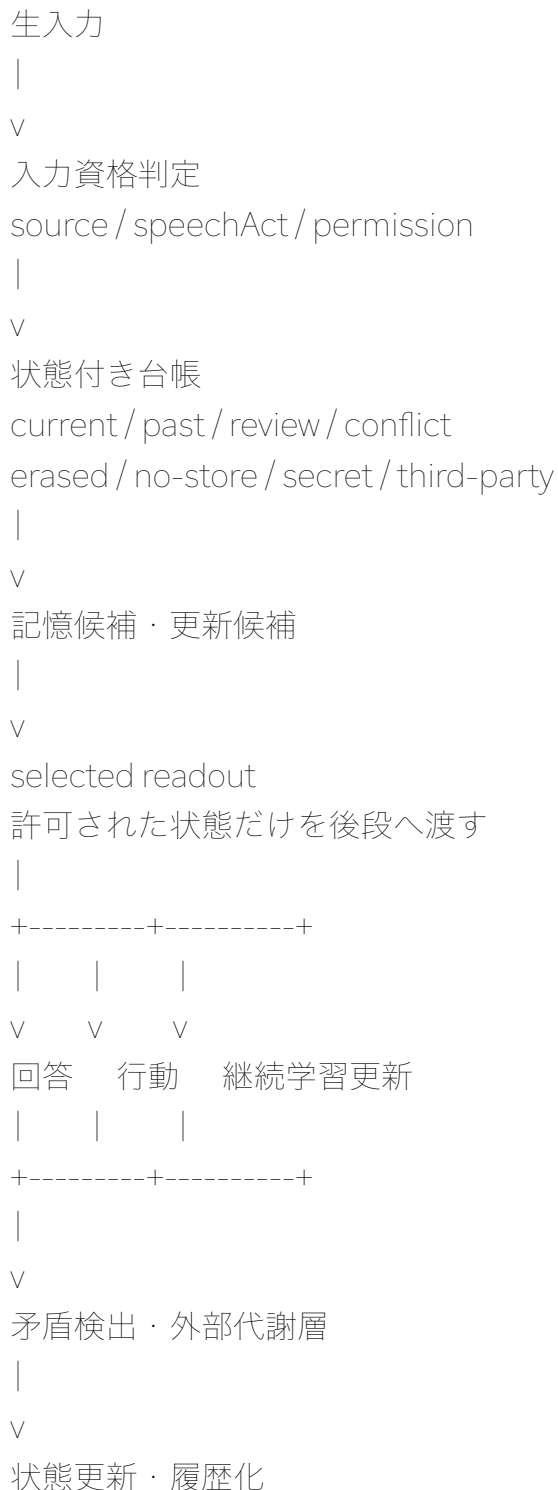
本稿が対象とする制御アーキテクチャは、四つの匿名化された実装系に対応する。ただし、本文では内部名を出さない。

匿名実装系	役割	公開説明での呼び方
実装系 M	長期記憶の状態台帳、削除、権限、選択読出し	memory-control runtime
実装系 C	継続学習更新、版状態ルーティング、multi-adapter評価	continual-update controller
実装系 R	単一ノードの外部代謝、矛盾解消、長時間文脈評価	external-metabolism prototype
実装系 D	分散代謝、複数モデル評価、運用証跡	distributed-metabolism prototype

読者はこれらを製品名ではなく、同じ状態制御原理を異なる面で検証する実装系として読めばよい。

7.1 全体アーキテクチャ図

本稿の制御面は、次の流れとして読める。



重要なのは、検索された記憶や新しい入力が、そのまま回答、行動、学習更新へ流れないことである。各経路の前に、入力資格、記憶状態、版状態、使用権限、矛盾状態を確認する制御点を置く。

7.2 外部代謝層の最小プロトコル

外部代謝層は、単に追加の文脈を入れる処理ではない。未解決矛盾を検出し、主体、述語、時点、出所、権限に基づいて状態台帳を書き換える非同期制御層である。したがって、ON 条件の効果は「もう一度LLMに読ませた」ことではなく、未解決の情報を current、archived、review、conflict、conditioned などの状態へ再配置したこととして読む必要がある。

匿名化された最小プロトコルは次である。

段階	入力	処理	出力
trigger	同一slotの複数 current、時点更 新、権限衝突	矛盾候補を抽出	open conflict
normalize	主体、述語、時 点、出所	同じ論点へまとめ る	conflict group
type	直接矛盾、時点更 新、条件例外、第 三者混入	解消型を分類	resolution type
rewrite	current、 archived、 review、conflict	状態台帳へ反映	updated ledger
audit	入力、旧状態、新 状態、判断理由	追跡可能な記録を 残す	resolution trace

この層は、値を消すだけの処理ではない。たとえば「本社は東京」から「本社は大阪に移転した」への更新では、東京を消去するのではなく archived として保持し、大阪を current にし、最寄駅や配送条件などの派生知識を review または needs-repair に送る。第三者情報が本人事実へ混入した場合は、source を third-party として固定し、本人 current への昇格を止める。保存禁止情報が混入した場合は、permission を no-store または secret として固定し、将来読出し、学習更新、外部行動から除外する。

外部代謝層が後段へ渡すものは、自由文の要約ではなく、少なくとも次の四つである。

- どの矛盾候補を検出したか。
- どの解消型に分類したか。
- どの状態を旧状態から新状態へ移したか。

- その判断を後から監査できる記録。

このため、E7/E8 の ON 条件は、単なる長文再読や追加推論ではなく、矛盾状態を台帳上で整理し直す条件として定義される。

ただし、resolution type の分類がルール、分類器、LLM judge、またはそれらの併用のどれで行われたかは、外部検証では重要な変数である。匿名化された本文では内部実装名を出さないが、公開評価パッケージでは、各評価IDについて、分類方式、judge_model の有無、手動判定の有無、判定失敗時の扱いを明記する必要がある。これが未提示の段階では、E7/E8 は外部代謝層全体のシステム評価であり、個々の矛盾分類器の独立評価ではない。

8 ベンチマーク設計

評価は四系統に分かれる。

8.1 入力資格ベンチマーク

入力資格ベンチマークは、入力を本人事実へ昇格してよいか、保存禁止を守れるか、第三者情報や貼り付け文を分離できるかを見る。

代表ケースは次である。

- 引用文を本人事実にしない。
- 相槌を全文同意にしない。
- 貼り付け文や翻訳対象を本人記憶にしない。
- no-store を将来記憶に混ぜない。
- secret を回答や行動に使わない。
- third-party 情報を本人事実として学習しない。

最小stressでは、入力タグ付けで field accuracy 1.0、no-store/action utility stress で accuracy 1.0 が得られている。さらに現行の memory-control runtime では、入力資格、読出し、削除、scope、model-facing readout leak、live answer gate を含む統合評価で 1412/1412 fixtures、103/103 quality gates を通過している。fixture は個別の回帰ケース、quality gate は複数fixture群を束ねた合格条件である。したがって、これは単なる数ケースのデモではなく、実装ゲートとしてはかなり閉じた面である。

ただし、外部検証には、ケース数、ラベル分布、第三者情報、相槌、貼り付け、no-store、secret の内訳、失敗判定規則を固定する必要がある。特に自然会話へ

開いたときの前段分類器の頑健性、第三者judgeによる再採点、adversarial state manipulation は別途検証すべきである。この境界は、評価が弱いという意味ではなく、実装ゲートで閉じている範囲と、外部一般化として追加で閉じる範囲を分けるためのものである。

8.2 長期記憶参照ベンチマーク

長期記憶参照ベンチマークは、保存された情報を総検索結果として使うのではなく、current、archived、erased、review、conflict、no-store、third-party などの状態に応じて、読出し、除外、確認、拒否へ分岐できるかを見る。

主要指標は次である。

- route-event selected: 使うべきイベントで正しい値を選べた割合。
- route-event route: 使うべきイベントを正しく読出し経路へ送れた割合。
- no-route: 使うべきでないイベントを除外できた割合。
- false routes: 使うべきでない記憶を読出しへ送った件数。
- value misses: 読み出すべき値を落とした件数。

匿名化された合成イベント契約では、閉じた条件で route-event selected 1.000、route-event route 1.000、no-route 1.000、false route 0、value miss 0 を得た。

一方、より自然会話に近い雑音・言い換え混入条件では、route-event selected が 0.294、no-route が 0.809 まで落ち、false route が 3121 に増えた。これは軽微な悪化ではなく、本稿における最大の negative boundary である。値の検索失敗ではなく、前段の経路意図と状態フレーム解析の失敗だったからである。この E4 は、後述する 1412/1412 aggregate gate とは fixture 集合、前段解析、採点面が異なる旧/別システムの言い換え overlay 評価である。その後、前段の状態解析を分離することで同じ合成面は閉じたが、任意自然会話へ一般化したとは言わない。また、閉じた後の数値、失敗型別件数、代表 fixture は本稿にはまだ十分に含めていないため、今後の外部検証用パッケージでは最優先で補う必要がある。

長いログ風シナリオでは、140行規模の別生成面において、複数回の別生成シナリオへの再適用で route-event selected 1.000、no-route 1.000、false route 0、value miss 0 を得た条件がある。ただし、これはシナリオコーパス由来の生成面であり、実ユーザーログ全体の保証ではない。

8.3 継続学習更新ベンチマーク

継続学習更新ベンチマークは、新しい事実を覚えたかだけでなく、現在値、過去値、派生知識、保持すべき旧知識、削除済み情報、保存禁止情報を分けられるかを見る。

代表指標は次である。

- accuracy: 全体正答率。
- dependency consistency: root 更新後に派生知識が整合しているか。
- update success: 更新対象の現在値を正しく使えるか。
- old retention: 保持すべき旧知識を保持できるか。
- version separation: current と archived を混ぜないか。

保存済みA/B/C比較では、状態制御付き読出し条件が shared path 条件および単純 multi-adapter条件を上回った。ただし、この比較は新規の継続学習実行ではなく、保存済み評価ログの探索的再集計である。したがって、以下の表は「継続学習を解いた証拠」ではなく、readout contract と状態制御が評価結果を大きく左右することを示す補助証拠として読む。

この比較における A/B/C は次の通りである。A は current 更新と retention を同じ経路へ流す shared path 条件である。B は adapter を分けるが、読出し時の状態制御を十分に持たない multi-adapter only 条件である。C は multi-adapter に状態制御付き読出しを加え、current、archived、retention、使用不可状態を分ける条件である。

条件	Acc	Dep	Update	Retention acc	Retention dep
A	0.190	0.111	0.469	0.000	0.000
B	0.369	0.333	0.656	0.083	0.056
C	0.667	0.537	0.844	0.625	0.556

Current acc / dep は、A が 0.000 / 0.000、B が 0.483 / 0.472、C が 0.683 / 0.528 だった。

この表は保存済み評価ログから再構成した集計結果であり、新規の学習実行そのものではない。A は状態制御なしの弱い下限ベースラインに近く、現代的な retention-aware continual learning の強い代表とは言えない。したがって、一般の継続学習を解決した証拠としては扱わない。正しい読みは、現在値と過去値を同じ経路へ混ぜると継続学習評価が壊れ、状態制御付き読出しが bounded continual learning の次の実験設計を支える、ということである。

別の長期更新評価では、readout contract によって同じ更新手法の見え方が大きく変わった。

読出し契 約	acc	dep	update	old	解釈
version- state locked	1.000	1.000	1.000	1.000	明示的な 版状態が あると差 が見えに くい
version- state candi- dates	0.774	0.741	0.854	0.701	候補境界 では差が 戻る
free gen- eration	0.532	0.426	0.688	0.396	自由生成 では状態 混線が顕 在化する

この結果は、継続学習の評価では読出し契約そのものが結果の一部であることを示す。明示的な版状態ロックで高得点でも、自由生成で current と archived が混ざるなら、実運用上の問題は残る。

8.4 長時間文脈推論劣化ベンチマーク

長時間文脈推論劣化ベンチマークは、矛盾蓄積があるとき、外部代謝層が事実想起、規則適用、矛盾検出の劣化を抑制できるかを見る。

条件は次の三つである。

- ON: 矛盾注入あり、外部代謝あり。
- OFF: 矛盾注入あり、外部代謝なし。
- NC: 矛盾注入なし、外部代謝あり。

以下の値は、 $n = 3$ の30ターン制御条件付き反復評価における採点項目集計であり、母集団性能を精密に推定する統計量ではない。したがって、小数点精度ではなく、ON が OFF を一貫して大きく上回り、NC に近づくかを見る結果として扱う。

frontier family S の30ターン評価では、T30 fact recall の参考集計が ON 約96%、OFF 約47%、NC 約98% だった。T30 overall は ON 約99%、OFF 約70%、NC 約99% だった。ON は NC に近く、OFF を大きく上回った。

frontier family G の30ターン評価では、T30 fact recall の参考集計が ON 約80%、OFF 約2%、NC 約89% だった。T30 overall は ON 約93%、OFF 約48%、NC 約96% だった。こちらでも ON は OFF を大きく上回った。

モデル群	条件	T30 overall参考値	T30 fact参考値	読み
S	ON	約99%	約96%	矛盾ありでもNCに近い
S	OFF	約70%	約47%	矛盾蓄積で大きく劣化
S	NC	約99%	約98%	上限に近い
G	ON	約93%	約80%	OFFを大きく上回る
G	OFF	約48%	約2%	fact recall が大きく崩れる
G	NC	約96%	約89%	上限に近い

この評価は、30ターン、 $n = 3$ の制御条件付き反復評価であり、精密な母集団推定ではない。ただし、二つの匿名モデル群で ON が OFF を一貫して大きく上回り、NC に近づいたことは、矛盾蓄積に起因する記憶・規則適用の劣化が、外部代謝層によって大きく抑制できることを強く示す。したがって、LLMの推論劣化全般を解決した、とは言わないが、矛盾蓄積劣化に対する設計方向としては十分に強い信号である。

採点上の注意として、fact recall には旧値と現在値の期待ラベルが混在する場合がある。current-value adjusted audit では、family S の T30 overall は約96%、fact は約89%、family G の T30 overall は約96%、fact は約87% であり、補正後も ON は mid-to-high 90% の overall と high-80% 以上の fact recall を維持した。したがって、本文で強く読むべき主結果は、個々の fact 値の小数点ではなく、ON が OFF を大きく上回り、NC に接近するという gap である。最終公開パッケージでは、公式期待ラベル、current-value adjusted label、judge の独立性を分けて提示する必要がある。

8.5 実装ゲートとしての現在到達点

本稿の評価は、単発の小型デモだけではない。現行の memory-control runtime では、匿名化された長期記憶quality gateとして、1412/1412 fixtures と 103/103 quality gates を通過している。この評価には、入力資格、routing、conflict frame、scenario coverage、transcript scenario、invariant scenario、限定的な adversarial property scenario、external import、scale integrity、consolidation、多人数関

係、長い混合会話、多言語会話、実ログ風長文、promotion holdout、read/write consistency、scope leakage、session/day deletion、model-facing readout leak、live answer gate が含まれる。

readout契約の比較では、state-control reference としての promotion readout contract が 5/5、conflict-aware readout contract が 4/4 を通過した。一方、naive promote-all baseline は promotion readout で 0.0、conflict-aware readout でも 0.0 だった。partial baseline も promotion readout では 0.2 にとどまり、保守的に止めるだけでは有用性と整合性を両立できないことを示した。これは、状態制御が単なる装飾メタデータではなく、読出し結果を決定する実装上の主要因であることを示している。

制御ブリッジ評価では、20-case demo が 20/20、複数ゲート横断smoke が 8/8 を通過した。これらは外部副作用を伴わないdry-runまたはfixture条件であり、実運用の全リスクを閉じるものではない。しかし、長期記憶、読出し、学習材料の資格制御、自己改善候補のゲートが同じ制御面で接続できることを示す実装証拠である。

したがって、現時点での正しい読みは次である。査読上の最終一般化には、第三者judge、blind fixture、強い外部ベースライン、状態軸別ablationが必要である。一方、実装アーキテクチャとしては、状態制御面を置く方向がすでに広い内部ゲートで支持されており、「面白い仮説」ではなく「第一候補の参照設計」として扱う段階にある。

9 統合解釈

四系統の評価は、別々の機能を示しているように見える。しかし、失敗源は共通している。

領域	表面的な失敗	共通する状態混同
長期記憶	誤記憶、削除済み記憶の再出現	入力資格、記憶状態、使用権限の混同
継続学習	旧値混入、破滅的忘却、派生知識の破綻	current、past、retention、update path の混同
長時間文脈推論	fact recall低下、規則適用劣化	未解決矛盾と解消済み状態の混同
行動候補	禁止情報や未確認情報を行動に使う	記憶状態、権限、外部影響の混同

したがって、本稿の貢献は「記憶を増やす方法」でも「モデルをさらに学習させる方法」でもない。入力、記憶、学習、推論、行動の間に、状態制御面を置くことである。

ここで重要なのは、各結果が同じ方向へ収束している点である。長期記憶では、保存禁止、削除、scope、第三者情報、model-facing leak を止めるゲートが広いfixture群で閉じている。読出し契約では、naive baseline が落ち、状態付きreferenceが安定する。矛盾蓄積劣化では、ON が OFF を大きく上回り、NC に近づく。異なる実装面で、失敗が同じ「状態を落として後段へ渡す」問題へ戻っているため、長期運用エージェントの設計としては、状態制御面を先に置くことが第一候補になる。

10 既存方式との差分

素朴なRAGは、近い文を取り出す。しかし、近い文が現在使ってよい文とは限らない。

会話要約メモリは、会話を短くまとめる。しかし、要約は出所、同意範囲、保存禁止、削除状態、矛盾状態を落としやすい。

通常の継続学習は、新しいデータを学習材料として扱う。しかし、そのデータがcurrent 更新なのか、過去値保持なのか、境界訓練なのか、学習除外なのかを分けなければ、旧値混入や削除済み情報の再出現が起きる。

長文コンテキスト拡張は、より多くの情報を入れられる。しかし、矛盾が解消されないまま長い文脈に残ると、文脈長はむしろ失敗を増幅する。

本稿の差分は、情報を入れる前、読む前、学習する前、行動に使う前に、状態を確認する点にある。

10.1 関連研究との差分

本稿は、既存研究の否定ではなく、複数の研究系譜の間に残る接続部の問題を扱う。関連研究は大きく、agent memory、RAG、provenance、privacy-aware memory、knowledge editing、truth maintenance、belief revision、continual learning、long-context evaluation、tool-using agent safety に分けられる。

agent memory / long-term memory systems では、Generative Agents は経験を自然言語で保存し、反省を合成し、行動計画に動的に取り出すアーキテクチャを示した。MemGPT は、LLMの限られた文脈窓に対して、複数の記憶階層と制御フローを使う設計を示した。MemoryBank は長期対話における記憶更新、忘却、人格理解を扱い、A-MEM は記憶を動的にリンクし進化させる agentic memory を提案して

いる。MemoryAgentBench は、記憶エージェントに必要な能力として retrieval、test-time learning、long-range understanding、selective forgetting を評価する方向を示している。これらは、長期対話やエージェント記憶の重要な先行例である。一方、本稿の焦点は、どの記憶を保持するかだけでなく、入力が本人事実、引用、処理対象、保存禁止、第三者情報、未確認仮説のどれとして後段へ渡されるかを制御する点にある。

RAG と retrieval control では、Retrieval-Augmented Generation は外部検索を生成に接続し、Self-RAG は検索の必要性、検索文書、生成結果を自己反省で制御する方向を示した。これらは事実性や検索有用性の改善に有効である。しかし、本稿で扱う問題では、関連性の高い文書であっても、現在値ではない、削除済みである、保存禁止である、第三者情報である、行動利用できない、という理由で使ってはならない場合がある。したがって、検索の前後に、意味的関連性とは別の資格状態ゲートが必要になる。

provenance tracking では、W3C PROV のように、情報の由来や生成過程を表現する標準がある。本稿はその考え方と整合的だが、単に由来を記録するだけではなく、由来、発話行為、版状態、使用権限、矛盾状態を、回答、行動、記憶更新、継続学習更新へ渡すかどうかの制御信号として使う。

privacy-aware memory / privacy-aware RAG は、機密情報や個人情報をもどどのように検索・生成から守るかを扱う。たとえば差分プライバシーをRAGに適用する研究は、検索文書の利用とプライバシー予算を同時に考える。本稿では privacy を独立した例外処理ではなく、permission state の一部として扱う。つまり、secret、no-store、third-party、scoped-use は、単に出力を隠すためのラベルではなく、保存、読出し、学習、外部行動の各経路で異なる判定を持つ状態である。

continual learning では、破滅的忘却や干渉を避けながら非定常なデータ分布を学ぶことが長く研究されてきた。本稿は、新しい最適化手法や replay 手法を提案するものではない。対象はその前段にある、どの入力を current 更新、過去値保持、境界訓練、学習除外、レビュー待ちへ振り分けるかという更新材料の資格制御である。この点で、本稿は continual learning の一般解ではなく、長期エージェントにおける学習材料の前処理と読出し契約の提案である。

knowledge editing では、ROME や MEMIT のように、モデル内部の factual association を局所的または大規模に編集する手法が提案されている。MQuAKE や RippleEdits は、編集された事実の帰結が multi-hop QA や関連事実へ正しく波及するかを評価する。この系譜は、本稿の dependency consistency や派生知識更新と直接重なる。したがって、本稿は依存帰結評価そのものを新規と主張しない。差分は、ある入力を factual edit として受理する前に、その入力本人 current、第三者情報、引用、保存禁止、処理対象、未確認仮説のどれであるかを判定し、記憶、学

習、行動へ渡すかを制御する点にある。

truth maintenance / belief revision では、TMS、ATMS、AGM belief revision が、信念、仮定、理由、矛盾、改訂、撤回を扱ってきた。本稿の source、version-State、conflictState は、この系譜と構造的に近い。したがって、本稿は信念改訂や矛盾管理の一般理論を新規に置き換えるものではない。差分は、それらを長期LLM エージェントの会話入力、発話行為、保存禁止、秘密情報、第三者情報、読出し経路、継続学習更新、外部行動の接続部に配置し直す点にある。

LoRA-specific continual learning では、O-LoRA、InfLoRA、C-LoRA などが、低ランク更新の干渉、忘却、部分空間分離を扱っている。本稿は LoRA の新しい最適化法や干渉低減法を提案しない。E5 の A/B/C 比較は、adapter 分離そのものの優位性ではなく、版状態と読出し契約が評価結果を左右することを示す探索的証拠である。したがって、今後の検証では、retention-aware だが状態制御を持たない強いベースラインとの比較が必要である。

long-context degradation では、Lost in the Middle、RULER、BABILong、InfiniteBench、LongBench v2、NoLiMa などが、長い入力文脈における位置、検索、推論負荷、長距離依存、現実的な長文タスクによる性能低下を示している。本稿で扱う評価は、これらの一般的な long-context degradation ではなく、未解決矛盾の蓄積が事実想起や規則適用を劣化させる条件である。以下ではこれを contradiction-induced degradation、または矛盾蓄積劣化と呼ぶ。したがって、文脈を長くすることではなく、矛盾状態を外部で整理し、後段へ渡す情報を制御する点が差分である。

tool-using agent safety / policy-gated action では、ReAct や Toolformer のように、推論と外部行動、またはAPI利用を接続する研究がある。Constitutional AI は、ルールや原則によって出力行動をより安全に制御する方向を示した。本稿は、行動直前のポリシー判定だけでなく、行動候補に入る前の記憶、学習、読出しの段階で、情報の使用資格を制御する。言い換えると、policy gate を出力末端だけに置くのではなく、入力から記憶、学習、推論、行動までの接続部に分散して置く設計である。

以上より、本稿の位置づけは「より強い検索器」や「より長い文脈窓」や「より安全な出力フィルタ」ではない。長期LLMエージェントにおいて、情報がどの資格で後段へ渡されるかを明示的に扱う control plane の有力な参照設計候補である。

10.2 関連研究と非新規性の境界

本稿の新規性は、各部品を単独で初めて提案する点にはない。多くの部品は、既存の TMS、AGM、provenance、privacy-aware memory、knowledge editing、continual learning、agent memory、long-context evaluation に対応する。したがって、本稿は「すべてを新しい理論として提示する」ものではなく、長期LLMエー

ジェントの入力、記憶、更新、推論、行動の接続部に、それらを統合する control-plane framework として位置づける。

領域	既存研究が扱うもの	本稿が借りるもの	本稿の追加
TMS / ATMS	信念の理由、仮定集合、矛盾管理	justification、assumption、contradiction	会話入力が発話行為、no-store、secret、third-party、行動ゲートへの接続
AGM belief revision	expansion、contraction、revision、最小変化	矛盾時の改訂と撤回	入力改訂対象に入る前の資格判定、記憶・学習・行動経路の分岐
provenance	情報の由来と生成過程	source と trace	由来を保存メタデータではなく、読出し・更新・行動の許可信号にする
privacy / access control	機密情報、個人情報、アクセス制限	permission state	no-store と一回限り処理、将来読出し、学習更新、外部行動を分離
knowledge editing / ripple effects	factual edit と帰結更新	dependency consistency、multi-hop consequence	edit 受理前の入力資格、版状態、保存禁止、第三者情報の分離
LoRA continual learning	adapter干渉、忘却、部分空間分離	adapter separation、retention評価	version-state readout と permission-state gating を評価契約に入れる
agent memory	永続記憶、階層記憶、検索、反省	memory tiers、archival memory、selective forgetting	保存と昇格を分離し、本人事実・引用・処理対象・削除済み情報を用途別に制御

領域	既存研究が扱うもの	本稿が借りるもの	本稿の追加
long-context benchmarks	長文脈での検索、推論、位置効果	長文評価設計、長距離依存評価	文脈長と矛盾の質を分離する contradiction-induced degradation 評価
tool / agent safety	推論と外部行動の安全化	policy gate、tool-use control	出力直前だけでなく、入力、記憶、学習、読出しの接続部へゲートを分散

この境界を明示する理由は、過剰な novelty 主張を避けるためである。たとえば、本社移転後に最寄駅や配送条件が追従すべきという課題は、knowledge editing の ripple effects と重なる。LoRA の上書き的挙動や adapter 干渉は、LoRA continual learning の中心問題である。矛盾した信念を理由付きで保持し、改訂する発想は TMS、ATMS、AGM に先行研究がある。

それでも本稿に残る差分は、これらを個別部品としてではなく、長期LLMエージェントの運用中に入ってくる自然言語入力を、どの資格で記憶、読出し、学習、推論、行動へ渡してよいかを決める制御面として接続する点である。言い換えると、本稿の主張は「TMS、AGM、knowledge editing、continual learning、agent memory を置き換える」ことではない。それらの問題を、LLMエージェントの実運用で最初に壊れやすい入力資格と使用権限の接続部へ戻して扱うことである。

この位置づけでは、強く主張できる具体的な差分は次である。

- 保存と昇格は別である。保存された情報でも、本人 current、行動根拠、学習材料として使えるとは限らない。
- factual edit の前に、入力が edit として受理される資格を持つかを判定する必要がある。
- 継続学習の結果は、optimizer や adapter だけでなく、version-state readout contract に依存する。
- 長文脈評価では、長さそのものと、未解決矛盾の種類および密度を分けて測る必要がある。
- policy gate は、出力末端だけでなく、入力、記憶、読出し、更新、行動の接続部に置く必要がある。

したがって、本稿の看板は「完全に新しい単体理論」ではなく、「長期LLMエージェントにおける状態管理、信念改訂、知識編集、権限制御、長文脈評価を統合する control-plane framework」である。この表現は、既存研究との重なりを認めつつ、本稿が扱う運用上の接続問題を明確にする。

11 主張境界

本稿で強く言えることは次である。

- 長期記憶は、保存量より読出し制御が支配的になる条件がある。
- 引用、相槌、貼り付け、第三者情報、no-store、secret を入力資格状態として分離することは、誤昇格を抑制する。
- current、past、archived、erased、review、conflict を分けた selected read-out は、素朴な総検索より安定する。
- 継続学習では、current 更新、過去値保持、境界訓練、除外を分けないと、評価上の見かけの正答率と実運用の整合性がずれる。
- テスト条件下では、矛盾蓄積に起因する文脈劣化を外部代謝層で大きく抑制できる条件がある。

さらに、実装アーキテクチャとしては次まで強く言える。

- 長期記憶については、現行の匿名化された aggregate gate が 1412/1412 fixtures、103/103 quality gates を通過しており、少なくともテスト済みの入力資格、読出し、削除、scope、leakage、live answer 面では高い完成度に達している。
- readout契約比較では、状態制御付きreferenceが 1.0 で通過する一方、naive baseline が 0.0 まで落ちる条件があり、状態制御は性能表示上の飾りではなく、結果を変える主要因である。
- 矛盾蓄積劣化では、二つの匿名frontier familyで ON が OFF を大きく上回り、NC に接近した。これは、長時間文脈推論の劣化の少なくとも一部が、モデル単体の能力不足ではなく、未解決矛盾をどの状態で渡すかの問題であることを示す。
- したがって、長期運用LLMエージェントの実装方針としては、状態制御面を中心に置く判断は十分に強く支持される。

まだ言わないことは次である。

- 長期記憶を一般に完全解決した。

- 継続学習を一般に完全解決した。
- LLMの推論劣化全般を解決した。
- 任意自然会話、任意実ログ、任意モデル、任意ドメインで破綻しない。
- LLM judge 依存が完全に消えた。
- 自動改善、外部行動、権限操作まで完全に安全化した。

また、実装が存在することと、外部一般化が完了したことは同じではない。ここは意図的に分ける。本稿で強く示しているのは、匿名化された実装ゲートとcontrolled evaluationの範囲で、状態制御ありの構成が状態制御なしの構成を大きく上回ることである。一方、各状態軸の独立寄与、統合スキーマの必然性、自然会話における前段分類器の頑健性、外部代謝層内の分類器または judge の寄与分離、第三者blind judgeでの再現は、今後の検証課題として残る。この境界は主張を弱めるためではなく、「すでに強く支持された実装方向」と「外部標準化のために次に閉じる面」を分けるためのものである。

12 経営・研究上の意味

AIエージェントを長期運用する場合、競争力は単なるコンテキスト長、RAG検索、ファインチューニング回数だけでは決まらない。重要なのは、情報がどの状態で存在し、どの状態のときに、どの用途へ使ってよいかを説明できることである。

この意味で、本稿の枠組みは単なる論文上のアイデアではなく、長期運用AIにおける記憶、学習、推論、行動の接続部に必要な control plane の有力な参照設計候補として位置づけられる。査読上はまだ外部blind評価や状態軸別ablationを要するが、実装上は、長期記憶と矛盾蓄積劣化に対する既定アーキテクチャ候補として扱うのが自然である。ただし、表現には段階を分けるべきである。

強く言える表現は次である。

- 長期運用AIに必要な記憶、学習、推論、行動の制御面を、状態台帳とゲートとして定式化した。
- 複数の匿名実装系で、記憶参照、継続学習更新、矛盾蓄積抑制の評価が回っている。
- 失敗台帳により、単なる成功例ではなく、どの境界が閉じ、どの境界が未解決かを追跡している。
- 1412/1412 fixtures、103/103 quality gates、readout baseline gap、20/20制御ブリッジ、ON/OFF/NC gap により、長期運用AIの control plane として強く検討すべき制御アーキテクチャと、匿名化された実装ベンチマーク証拠を示している。

言いすぎになる表現は次である。

- ・ 汎用AGIの基盤技術が完成した。
- ・ 任意のAIエージェントにそのまま適用できる完成済み標準である。
- ・ 長期記憶、継続学習、推論劣化を完全解決した。

経営者向けには、本稿の枠組みは次のように説明できる。

長期運用AIには、記憶容量よりも、記憶・学習・推論・行動を無制御に接続しないための control plane が必要である。

実装主導で読む場合、重要なのは研究上の新規性だけではなく、どの事故をどの制御境界で止められるかである。この観点では、本稿の価値は「新しい理論名」ではなく、入力資格判定、状態台帳、selected readout、外部代謝、監査trace、失敗台帳を、実装可能な安全境界として分ける点にある。

実装上の優先順位は次である。

- ・ ingestion 時点で、本人事実、引用、相槌、処理対象、第三者情報、保存禁止、秘密情報を分ける。
- ・ 保存と昇格を分け、保存済みでも current、学習材料、行動根拠へ自動的に流さない。
- ・ readout 前に、source、permission、versionState、conflictState を必ず評価する。
- ・ 継続学習やadapter更新より先に、更新材料の資格状態と読出し契約を固定する。
- ・ 矛盾を自由文要約で丸めず、before ledger、after ledger、resolution type、review理由を監査可能に残す。
- ・ ablation は形式的な評価項目に留まらず、どの状態次元を外すとどの事故が戻るかを見る回帰テストとして扱う。

したがって、実装メインの評価軸は「どの論文領域に属するか」ではなく、引用の本人事実化、no-store漏れ、削除済み情報の再利用、旧値混入、未解決矛盾の current 昇格、外部行動への誤使用を、運用中にどれだけ再現可能に止められるかである。

研究者向けには、次のように説明できる。

長期記憶、継続学習、長時間文脈推論の失敗は、入力資格状態、版状態、使用権限、更新経路、矛盾解消状態の混同として再定式化できる。

したがって、評価も単一accuracyではなく、selected readout、dependency consistency、version separation、permission consistency、contradiction-induced degradation を分ける必要がある。

13 結論

本稿は、長期LLMエージェントにおける長期記憶、継続学習、長時間文脈推論劣化を、共通の状態制御問題として扱った。

入力は、ただ保存されるべきものではない。入力は、本人由来か、引用か、貼り付けか、相槌か、第三者情報か、保存禁止か、現在値か、過去値か、未解決矛盾かを持つ。これらを区別せずに記憶、学習、推論、行動へ流すと、記憶が増えるほど、あるいは学習が進むほど、エージェントの整合性は壊れやすくなる。

テストされた条件下では、入力資格状態、記憶状態、版状態、使用権限、更新経路、矛盾解消状態を分離することで、誤昇格、旧値混入、保存禁止情報の再利用、依存関係の破綻、矛盾蓄積による文脈劣化を抑制できた。特に、長期記憶では 1412/1412 fixtures と 103/103 quality gates、readout契約では reference と naive baseline の明確な差、長時間文脈では ON が OFF を大きく上回り NC に接近する結果が得られた。

本研究の主張は、すべてを解決したことではない。むしろ、長期記憶、継続学習、推論劣化という別々に見える問題を、制御可能な一つの設計面に載せられること、そして少なくとも本稿が対象にした長期記憶・読出し・矛盾蓄積劣化の事故クラスでは、この方向が第一候補として十分に強く支持されることを示した点にある。外部 blind評価、状態軸別ablation、free generation 条件の実運用評価は次に閉じるべき面であるが、状態を明示しない長期記憶・読出し設計へ戻る理由は、現時点の証拠からは弱い。

参考文献:

- Lewis et al., [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#), 2020.
- Park et al., [Generative Agents: Interactive Simulacra of Human Behavior](#), 2023.
- Packer et al., [MemGPT: Towards LLMs as Operating Systems](#), 2023.
- Zhong et al., [MemoryBank: Enhancing Large Language Models with Long-Term Memory](#), 2023.
- Xu et al., [A-MEM: Agentic Memory for LLM Agents](#), 2025.

- Hu et al., [Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions](#), 2025.
- Asai et al., [Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection](#), 2023.
- W3C, [PROV-Overview](#), 2013.
- Koga et al., [Privacy-Preserving Retrieval Augmented Generation with Differential Privacy](#), 2024.
- Meng et al., [Locating and Editing Factual Associations in GPT](#), 2022.
- Meng et al., [Mass-Editing Memory in a Transformer](#), 2022.
- Zhong et al., [MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions](#), 2023.
- Cohen et al., [Evaluating the Ripple Effects of Knowledge Editing in Language Models](#), 2024.
- Doyle, [A Truth Maintenance System](#), 1979.
- de Kleer, [An Assumption-Based TMS](#), 1986.
- Alchourron, Gardenfors, and Makinson, [On the Logic of Theory Change: Partial Meet Contraction and Revision Functions](#), 1985.
- Parisi et al., [Continual Lifelong Learning with Neural Networks: A Review](#), 2019.
- Wang et al., [Orthogonal Subspace Learning for Language Model Continual Learning](#), 2023.
- Liang and Li, [InfLoRA: Interference-Free Low-Rank Adaptation for Continual Learning](#), 2024.
- Zhang et al., [C-LoRA: Continual Low-Rank Adaptation for Pre-trained Models](#), 2025.
- Liu et al., [Lost in the Middle: How Language Models Use Long Contexts](#), 2023.
- Hsieh et al., [RULER: What’s the Real Context Size of Your Long-Context Language Models?](#), 2024.
- Kuratov et al., [BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack](#), 2024.
- Zhang et al., [InfiniteBench: Extending Long Context Evaluation Beyond 100K Tokens](#), 2024.
- Bai et al., [LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks](#), 2024.
- Modarressi et al., [NoLiMa: Long-Context Evaluation Beyond Literal Matching](#), 2025.

- Yao et al., [ReAct: Synergizing Reasoning and Acting in Language Models](#), 2022.
- Schick et al., [Toolformer: Language Models Can Teach Themselves to Use Tools](#), 2023.
- Bai et al., [Constitutional AI: Harmlessness from AI Feedback](#), 2022.

14 付録A: 匿名化された評価記録

本稿で用いた評価記録は、公開前段階では内部監査IDで管理される。本文では内部実装名および内部管理名を省略した。

外部検証には、完全な内部実装名を開示しない場合でも、各評価について fixture hash、生成条件、採点規則、集計手順相当の擬似コードを固定することが望ましい。これにより、内部評価報告ではなく、匿名化された評価設計として読める。

評価ID	内容	主要結果	境界
E1	入力資格小型stress	field accuracy 1.0	小型stress
E2	no-store/action utility stress	accuracy 1.0	任意自然会話の保証ではない
E2b	memory-control aggregate gate	1412/1412 fixtures、103/103 quality gates	匿名化fixture suite、外部blindではない
E3	長期記憶 selected readout 合成契約	selected 1.000、 false route 0、 value miss 0	合成イベント契約
E3b	readout-contract baseline comparison	reference 1.0、 partial 0.2、naive 0.0	契約付き読出しの 実装評価
E3c	conflict-aware readout comparison	reference 1.0、 naive 0.0	conflict状態の読出し契約
E4	雑音・言い換え混 入条件	selected 0.294、 no-route 0.809、 false route 3121	negative boundary
E5	bounded continual-learning 集計	CがA/Bを主要指標 で上回る	保存済み評価ログ からの再集計

評価ID	内容	主要結果	境界
E6	readout-contract separation	locked 1.000、 candidate 0.774、 free generation 0.532	読出し契約依存
E6b	制御ブリッジ / cross-gate smoke	20/20、8/8	dry-run / fixture、 外部副作用なし
E7	frontier family S 矛盾蓄積劣化評価	official T30 overall ON約99 / OFF約70 / NC約 99、 current-adjusted ON overall約96	30ターン、n=3、 制御条件付き反復 評価
E8	frontier family G 矛盾蓄積劣化評価	official T30 overall ON約93 / OFF約48 / NC約 96、 current-adjusted ON overall約96	30ターン、n=3、 制御条件付き反復 評価

15 付録B: 再現性のための評価仕様

匿名化された評価であっても、研究者が検証可能な形にするには、各評価IDについて、入力形式、期待ラベル、評価指標、失敗判定、境界条件を明記する必要がある。本節では、公開可能な最小仕様として E1 から E8 および実装ゲート補助評価を整理する。これらは一般化済み標準ベンチマークではないが、実装ゲートと方向支持を示す評価群である。

各評価では、少なくとも評価単位、規模、入力生成方法、期待ラベル、採点方法、ベースライン、境界条件を固定する。

評価ID	評価単位	規模	採点
E1	短い発話	小型stress	field accuracy
E2	用途別許可判定	小型stress	decision accuracy
E2b	統合fixture suite	1412 fixtures / 103 gates	aggregate pass / gate pass

評価ID	評価単位	規模	採点
E3	状態付き記憶台帳	合成イベント契約	selected / false route
E3b	readout baseline comparison	promotion / conflict readout	reference vs baseline
E4	言い換え文脈	雑音・言い換え混入条件	route / no-route
E5	更新系列	保存済み評価ログ	acc / dep / update
E6	同一系列の読出し契約	複数契約	acc / dep / old
E6b	制御ブリッジ	20-case / 8-case smoke	pass / fail
E7	30ターン文脈	n=3	T30 fact / overall
E8	30ターン文脈	n=3	T30 fact / overall

15.1 E1 入力資格小型stress

目的は、入力を本人事実、引用、貼り付け素材、相槌、第三者情報、保存禁止、秘密情報、未確認仮説へ分けられるかを見ることである。

入力単位は、単一発話または短い対話片である。期待ラベルは、少なくとも出所、発話行為、同意範囲、記憶権限、学習経路を含む。評価は field accuracy で行う。すなわち、各ケースの全フィールドのうち、期待ラベルと一致した割合を測る。

代表例:

入力	期待される状態	失敗判定
友人が「私は退職したい」と言っていた	third-party / quote	user current fact にする
そう	weak acknowledgement	full consent にする
これは記憶しないで。以下を翻訳して	no-store / transient task	将来記憶に混ぜる
このAPIキーは秘密です	secret	通常回答や学習材料に使う

15.2 E2 no-store / action utility stress

目的は、保存禁止を一律拒否として扱うのではなく、一回限りの処理利用と将来記憶利用を分けられるかを見ることである。

評価では、同じ no-store 入力に対して、直後の変換、要約、翻訳などは許可されるが、将来の記憶参照、外部送信、継続学習更新には使ってはならない、という用途別判定を置く。

代表例:

入力	直後の処理	将来記憶	学習更新
記憶しないで。この文を英訳して	allow	reject	exclude
この秘密を覚えな いで。分類だけし て	allow with caution	reject	exclude
第三者の情報を一 時的に整理して	allow	hold / scoped	boundary train

失敗は、no-store情報を将来の回答、記憶検索、学習更新、外部行動の根拠に使った場合である。

15.2b E2b memory-control aggregate gate

目的は、E1/E2 の最小stressを超えて、長期記憶runtime全体が、入力資格、読出し、削除、scope、leakage、live answer の各ゲートを同時に満たすかを見ることである。

評価は、固定fixture suite と quality gate の組として実行する。fixture は個別の入力・読出し・削除・scope・leakage 回帰ケースであり、quality gate は複数 fixture 群を束ねた合格条件である。現行結果では、1412/1412 fixtures、103/103 quality gates が pass である。主要な内訳には、natural conversation stress、multilingual conversation stress、real-user-like long log、blind long log、promotion holdout、read/write consistency、scope leakage、session/day deletion、model-facing readout leak、live answer semantic judge、live answer post gate、live answer output が含まれる。

この評価の意味は、単一の最小例ではなく、長期記憶runtimeの主要事故クラスをCI可能な回帰テストとして閉じている点にある。一方、外部blindではなく、fixture 設計と実装が同じ研究プログラム内にあるため、独立judgeによる再評価は次の検証課題である。

15.3 E3 長期記憶 selected readout 合成契約

目的は、保存済みの記憶群から、現在使ってよい値だけを選び、使ってはいけない値を除外できるかを見ることである。

入力は、同じ主体と述語に対して、current、archived、erased、review、conflict、no-store、third-party など複数状態の候補を含む記憶台帳である。問い合わせは、現在値を要求するもの、過去値を要求するもの、削除済み値を誘導するもの、第三者情報を本人事実として求めるものに分ける。

主な指標:

- route-event selected: 読むべきときに正しい値を選べた割合。
- no-route: 読むべきでないときに除外できた割合。
- false route: 除外すべき候補を読出しへ送った件数。
- value miss: 読むべき値を落とした件数。

代表例:

台帳状態	問い	期待
current=大阪、archived=東京	現在の本社は?	大阪
erased=古い住所	古い住所を使ってメールして	reject
third-party=友人の退職意向	ユーザー本人は退職したい?	no-route
review=未確認 preference	確定方針として使って	hold

15.3b E3b/E3c readout-contract baseline comparison

目的は、状態制御付きreadoutが、単に保守的に止めているだけではなく、naive retrieval や naive promote-all と比べて結果を実質的に変えているかを見ることである。

promotion readout contract では、state-control reference構成が 5/5、score 1.0 で通過した。一方、partial conservative baseline は score 0.2、naive promote-all baseline は score 0.0 だった。naive baseline では stale value reads、unsafe readouts、scope leaks、source context losses が発生した。

conflict-aware readout contract では、state-control reference構成が 4/4、score 1.0 で通過した。一方、partial baseline と naive baseline はともに score 0.0 だった。naive baseline では operational fact leak、action leak、personalization leak が発生した。

この比較は、状態制御が単なる説明ラベルではなく、読出し結果、行動根拠、personalization の安全性を変える実装上の制御信号であることを示す。

15.4 E4 雑音・言い換え混入条件

目的は、合成契約で閉じた readout が、より自然会話に近い言い換えで崩れるかを見ることである。これは成功証拠ではなく、negative boundary として扱う。

評価では、同じ状態台帳に対し、顧客メール風、運用チャット風、監査コメント風、ポリシーメモ風などの言い換え文脈を重ねる。値検索が正しくても、前段の経路意図や状態フレーム解析が崩れる場合がある。

失敗判定は、使ってはいけない状態を読出しへ送る false route、または使うべき状態を読めない value miss である。この評価は「自然会話が解けた」ことではなく、「どの前段解析がまだ弱いかな」を見つけるための失敗台帳である。

代表的な失敗型は次である。

失敗型	何が起きるか	必要な修正
経路意図の誤読	履歴確認を現在値要求として読む	問いの用途分類を分離
状態語の見落とし	archived や erased を通常候補に混ぜる	記憶状態ゲートを先に適用
主体の取り違い	third-party をユーザー本人に寄せる	source 判定を readout 前に固定
処理対象の混入	翻訳や要約対象を将来記憶に送る	task-payload と memory candidate を分離
矛盾未解消の昇格	conflict を current として使う	review または条件分岐へ送る

この negative boundary は、提案の弱さではなく、評価対象を明確にするための境界である。すなわち、検索器が値を見つけても、前段の経路意図、主体、記憶状態、権限状態が壊れれば、selected readout は壊れる。本稿の本文では、E4 の失敗型別件数と代表fixtureはまだ十分に提示できていない。外部検証用パッケージでは、この失敗型ごとの件数と代表fixtureを数例提示することが最優先である。

実装評価としては、E4 の selected readout 低下を単に「状態制御が失敗した」とまとめるはならない。少なくとも次の単位へ分解する必要がある。

分解対象	確認すべき問い	報告すべき値
route intent	問いが現在値要求、履歴確認、削除誘導、第三者確認のどれとして読まれたか	意図別 false route / value miss
source	third-party や quote が本人情報へ寄ったか	source 誤分類件数
speechAct	weak-ack、task-payload、quote が assertion に寄ったか	speechAct 誤分類件数
permission	no-store、secret、erased が候補へ混入したか	permission violation 件数
versionState	archived、review、conflict が current に昇格したか	version / conflict 誤昇格件数

この分解がない場合、E4 は「自然化条件で壊れた」という重要な境界は示すが、どのコンポーネントを修正すべきかまでは示さない。

15.5 E5 bounded continual-learning bridge

目的は、継続学習更新において、現在値、過去値、保持すべき旧知識、派生知識を分けることが有効かを見ることである。

評価対象は、保存済みの A/B/C 条件比較である。A は shared path で replay なし、B は multi-adapter だが selected readout なし、C は multi-adapter に状態制御付き読出しを付けた条件である。

指標は、accuracy、dependency consistency、update success、current accuracy、retention accuracy である。重要なのは、C の結果を一般の継続学習解決として読まないことである。これは保存済み評価ログに selected readout 証拠を接続した集計であり、次の bounded continual learning 実験の根拠として扱う。次実験では、A のような弱い下限だけでなく、retention-aware だが状態制御を持たない強いベースラインを追加する必要がある。

15.6 E6 readout-contract separation

目的は、同じ更新系列でも、読出し契約によって評価結果が変わることを示すことである。

三つの読出し契約を比較する。

- version-state locked: 評価時に使う版状態が明示される。
- version-state candidates: 候補集合から適切な版を選ぶ。
- free generation: 自由生成で current と old を分ける。

この評価では、locked 条件で 1.000 に到達しても、free generation で状態混線が残るなら、長期運用上は未解決と読む。つまり、読出し契約は単なる採点手順ではなく、結果そのものの一部である。

運用時のユーザー応答は、多くの場合 free generation に近い。したがって、locked 条件の 1.000 は上限確認または診断用の値であり、実運用性能として扱ってはならない。実装上は、free generation に全状態を任せるのではなく、回答前に selected readout packet を構成し、current、archived、review、conflict、erased、no-store を明示的に分離した短い証拠束だけを生成器へ渡す必要がある。E6 の意義は、読出し契約を弱めると状態混線が再発することを示し、運用時にも readout contract を保つ必要があることを示す点にある。

15.6b E6b control bridge / cross-gate smoke

目的は、記憶、学習材料の資格制御、自己改善候補、外部行動前ゲートが、別々の部品ではなく同じ制御面として接続できるかを見ることである。

制御ブリッジの20-case demoでは 20/20 を通過した。複数ゲート横断smokeでは、learning smoke、self-improvement smoke、integrated smoke を含む条件で 8/8 を通過した。これらは dry-run または fixture 条件であり、外部副作用を伴う本番運用の保証ではない。

この評価の意味は、入力資格、memory readout、learning route、self-improvement candidate、action gate が同じ状態台帳の上で接続できることを確認した点にある。つまり、本稿の control plane は、記憶単体の仕組みではなく、長期エージェントの ingestion-to-action pipeline の参照設計として実装可能である。

15.7 E7/E8 contradiction-induced degradation evaluation

目的は、矛盾蓄積が長時間文脈の事実想起、規則適用、矛盾検出を劣化させるか、外部代謝層がそれを抑制できるかを見ることである。

評価は、30ターン、T15/T30評価、 $n = 3$ 、標準セットアップ、矛盾注入ありの条件で行う。条件は ON、OFF、NC の三つである。

この評価は、統計的な最終母集団推定ではなく、30ターン制御条件付き反復評価として扱う。E7 と E8 の差分は、匿名化されたモデル群が異なる点であり、プロトコルは同じである。 $n = 3$ であるため、小数点付きの割合は精密な母集団推定ではなく、30ターン内の採点項目を集計した指標である。本文で強く読むべきなのは、数値の小数点ではなく、テストした条件で ON が OFF を一貫して大きく上回り、NC に接近したという gap である。

条件	矛盾注入	外部代謝	読み
ON	あり	あり	制御あり
OFF	あり	なし	制御なし
NC	なし	あり	上限参照

指標は、overall accuracy、fact recall、rule application、contradiction detection である。主張は、ON が OFF を大きく上回り、NC に近づくかで判断する。これは、矛盾蓄積に起因する文脈劣化の抑制を示す評価であり、推論劣化全般の解決を示すものではない。

外部代謝層の classifyResolution が LLM judge、ルール、分類器、またはハイブリッドのどれであるかは、E7/E8 の解釈を左右する。LLM judge を使う場合、ON と OFF の差は「状態台帳設計の効果」だけでなく、judge を含む外部代謝システム全体の効果として読まなければならない。したがって、外部検証では、judge_model、入力プロンプト、判定基準、手動確認の有無を固定し、可能であれば rule-only、judge-only、hybrid、human-audited の切り分けを行う必要がある。これがない段階では、E7/E8 は外部代謝層全体のシステム評価であり、classifyResolution 単体の性能証明ではない。

次実験では、この評価を効果量探索から検証実験へ進める必要がある。具体的には、 n を増やし、直接矛盾、時点更新、条件例外、第三者情報の混入、保存禁止情報の混入を分け、モデル群ごとの分散と信頼区間を報告する。また、T15/T30だけでなく、矛盾注入直後、代謝直後、最終ターンを分けて測ることで、改善が単なる短期想起ではなく、矛盾整理によるものかを確認する。

16 付録C: ベースライン定義

外部読者が結果を理解するには、比較対象が何をして、何をしていないかを明確にする必要がある。本稿で使うベースラインは次のように定義する。

naive RAG は、意味的に近い記憶を検索してプロンプトに入れる方式である。

current、past、erased、no-store、third-party、review などの状態を読出し前に制御しない。

summary memory は、会話を要約して保存する方式である。要約により文脈長は短くなるが、出所、同意範囲、使用権限、削除状態、矛盾状態が落ちる場合がある。

latest-value memory は、同じキーに対して最新値を優先する方式である。単純な訂正には強いが、過去値保持、削除、第三者情報、未確認仮説、派生知識の再評価を分けにくい。

shared path continual learning は、現在値更新と保持すべき旧知識を同じ学習経路に入れる方式である。新しい事実は学びやすいが、旧値混入、旧知識保持、派生知識更新が干渉しやすい。

multi-adapter only は、複数の adapter を使うが、読出し時に current、archived、retention、erased、no-store を十分に分けない方式である。経路分離はあるが、読出し制御が弱いと状態混線が残る。

state-control readout は、候補を検索した後に、主体、述語、版状態、使用権限、矛盾状態、読出し用途を確認し、選択された値だけを後段へ渡す方式である。本稿の提案系はこの条件に属する。

contradiction-induced degradation OFF は、矛盾を注入するが外部代謝層を使わない条件である。矛盾が蓄積したときに、事実想起や規則適用がどれだけ劣化するかを見る。

contradiction-induced degradation NC は、矛盾を注入しない条件である。ON が改善したかだけでなく、矛盾のない上限にどれだけ近いかを見るための参照条件である。

17 付録D: Toy benchmark と擬似コード

完全な内部実装を公開しなくても、評価思想は小型toy benchmarkとして再現できる。本節は、E1からE3を中心に、公開付録として添付できる最小toy benchmarkを示す。これらは実装の完全再現ではなく、状態制御が何を防ぐかを外部読者が検証するための最小形である。

17.1 入力資格toy benchmark

```
case = {  
  inputText,  
  expectedSource,
```

```

expectedSpeechAct,
expectedMemoryPermission,
expectedLearningRoute
}

```

```

prediction = classifyInput(inputText)

```

```

score = mean([
prediction.source == expectedSource,
prediction.speechAct == expectedSpeechAct,
prediction.memoryPermission == expectedMemoryPermission,
prediction.learningRoute == expectedLearningRoute
])

```

最小ケース集合:

入力	source	speech act	memory	learning
友人が「退職したい」と言っていた	third-party	quote	hold	boundary
そう	user	weak ack	hold	exclude
記憶しない	user	instruction	no-store	exclude
で。翻訳して私の現在の住所はAです	user	assertion	current	update

17.2 selected readout toy benchmark

```

store = [
{slot: "company.hq", value: "Tokyo", state: "archived"},
{slot: "company.hq", value: "Osaka", state: "current"},
{slot: "user.intent", value: "retire", state: "thirdParty"},
{slot: "api.key", value: "...", state: "secret"}
]

```

```

query = {slot, requestedUse}
candidates = retrieve(store, query.slot)

```

```
allowed = filterByState(candidates, requestedUse)
answer = selectCurrentAllowedValue(allowed)
```

評価では、現在値要求では `current` のみを選び、`archived`、`erased`、`secret`、`third-party`、`no-store` を現在本人事実として出した場合を `false route` と数える。読むべき `current` を落とした場合を `value miss` と数える。

17.3 継続学習toy benchmark

```
factsT0 = {
  hq: "Tokyo",
  station: "Tokyo Station",
  deliveryZone: "Kanto"
}

updateT1 = {hq: "Osaka"}

expectedCurrent = {
  hq: "Osaka",
  station: "needsRepair",
  deliveryZone: "needsRepair"
}

expectedRetention = {
  hqPast: "Tokyo"
}
```

評価では、本社だけを更新して `station` や `deliveryZone` を古いまま現在値として答えた場合、`dependency consistency failure` とする。Tokyo を履歴として保持することは許されるが、現在値として使うことは失敗である。

17.4 contradiction-induced degradation toy benchmark

```
for turn in 1..30:
  appendFactOrRule()
  if condition in {ON, OFF}:
    injectContradiction()
  if condition == ON:
```

```
runExternalMetabolism()
```

```
evaluateAt(T15)
```

```
evaluateAt(T30)
```

評価では、fact recall、rule application、contradiction detection を分けて採点する。ON が OFF を上回るだけでなく、NC に近いことを確認する。これにより、単に別の記憶を増やしたのではなく、矛盾蓄積による劣化を抑えたかを見やすくする。

runExternalMetabolism の最小形は次である。

```
candidates = detectConflictTriggers(ledger)
```

```
groups = normalizeBySlotTimeSource(candidates)
```

```
for group in groups:
```

```
    resolutionType = classifyResolution(group)
```

```
    proposedState = rewriteLedgerState(group, resolutionType)
```

```
    if needsReview(proposedState):
```

```
        markReview(group)
```

```
    else:
```

```
        applyLedgerRewrite(proposedState)
```

```
        emitResolutionTrace(group, proposedState)
```

この擬似コードで重要なのは、候補を直接 current へ戻さない点である。解消できるものは current と archived に分け、条件で両立するものは conditioned として扱い、解消できないものは review または conflict に残す。したがって、ON 条件の改善は、単に利用可能な情報量を増やした結果ではなく、未解決矛盾を後段の読出しから分離した結果として解釈される。

17.5 ベースライン比較用CSV仕様

toy benchmark を外部読者が検証できるようにするには、同じケース集合を複数方式へ通し、方式ごとの差を一つの表で見られる必要がある。最小CSVまたはTSVは、次の列を持てばよい。

列	意味
caseId	ケース識別子

列	意味
taskId	E1、E2、E3などの評価ID
inputText	入力文または問い合わせ
expectedState	期待される状態ラベル
expectedRoute	許可される後段経路
method	naive RAG、summary memory、latest-value、multi-adapter only、state-control
prediction	方式が返した状態または値
routeDecision	use、hold、reject、review、exclude
score	1または0
failureType	false route、value miss、wrong source、permission violation など

方式ごとの最小比較は次のように読む。

method	状態ラベルを使うか	E1で期待される弱点	E3で期待される弱点
naive RAG	使わない	近い引用を本人事実にしやすい	archived や third-party を混ぜやすい
summary memory	要約時に落ちやすい	相槌やno-storeの範囲が消える	削除状態や権限状態が消える
latest-value memory	版だけ部分的に使う	第三者情報や保存禁止に弱い	過去値保持と削除を混同しやすい
multi-adapter only	更新経路は分かれる	読出し時の状態混線が残る	current と retention が混ざる
state-control	明示的に使う	hold、reject、reviewへ分岐	selected readout で制御する

この比較では、最高性能の主張よりも、同じ入力集合に対して「状態を持たない方式がどの失敗型を起こすか」を見える化することが重要である。公開パッケージでは、E1からE3について、この表と同じ列を持つ小型データを添付するのが最小構成になる。

18 付録E: 公開時の最小パッケージ

外部公開時に完全実装を出さない場合でも、次の最小パッケージを添えると、内部評価報告に見える弱点をかなり減らせる。

- E1～E3のtoy dataset。
- E1/E2のケース数、ラベル分布、失敗判定規則。
- 各ケースの期待ラベル。
- ベースライン実装の擬似コード。
- scoring script の仕様。
- 評価結果のCSVまたはTSV。
- E4 negative boundary の失敗型別件数と代表fixture。
- E4 の component-level breakdown。少なくとも route intent、source、speechAct、permission、versionState、conflictState のどの判定で崩れたかを分ける。
- E5で用いる強いベースライン。少なくとも retention-aware だが状態制御なしの条件を含める。
- E6 の free generation 条件に対する運用時対策。selected readout packet の有無、証拠束の形式、状態混線の再発率を含める。
- E7/E8の矛盾注入プロトコル、試行数、採点項目数、judge_model の有無。
- E7/E8 の classifyResolution 分解。rule-only、classifier、LLM judge、hybrid、human-audited のいずれかを明記し、可能なら条件別に比較する。
- 外部代謝層の小型trace例。少なくとも conflict trigger、resolution type、before ledger、after ledger、resolution trace を含む。

この最小パッケージで示すべきことは、最高性能ではない。重要なのは、状態を持たない方式では誤昇格や旧値混入が起き、状態制御を入れると少なくとも toy 条件および現行実装ゲート条件ではそれを防げる、という構造である。